Plan 1 — Travell

Point-to-point swarm mission · domain `swarm-drone-mission` · plan kind `point_to_point`
A complete walkthrough of domain.pddl, problem.pddl and every Python function that runs them.

1. What this plan does

Travell is the simplest of the four missions. The operator clicks a **source** and a **destination** on the map, optionally marks a **restricted area** (a no-fly polygon), and ticks however many drones should fly. The whole ticked swarm then flies **one shared route** from the source to the destination - around the restricted area if, and only if, that area genuinely lies across the straight line between the two picked points.

The route itself is found by ENHSP, a numeric PDDL planner. The PDDL files in `plans/travell/` are not the mission that actually flies: they are a **template**. Every run rewrites that template onto the points the operator picked, expands it from one reference drone to N drones, hands the result to ENHSP, and turns ENHSP's action list back into a list of latitude/longitude waypoints.

1.1 The pipeline, end to end

The Travell planning pipeline

```
GUI: operator picks source + destination (+ optional no-fly polygon)
     and ticks N drones
                  |
                  v
services/plan_service.py   PlanService.run_async(...)
     -> _Worker.run(...)  [on a background QThread]
     -> _Worker._run(...)
                  |
                  +--> engine/pddl_problem.py  retarget_problem(...)
                  |       move the whole waypoint graph onto the picked
                  |       points; decide direct route vs. detour
                  |
                  +--> engine/pddl_problem.py  expand_point_to_point_drones(...)
                  |       drone1  ->  drone1 .. droneN
                  |
                  v
     plans/travell/runs/<timestamp>/problem.pddl   (the concrete problem)
                  |
                  v
engine/pddl_planner.py     run_enhsp(domain.pddl, problem.pddl)
     -> java -jar enhsp.jar -o domain -f problem   [subprocess]
     -> parse_plan(stdout)   ->  list[PlanStep]
                  |
                  v
     extract_route(steps, "drone1")   ->  ["source", "waypoint1", "destination"]
                  |
                  v
     read_locations(concrete_text)    ->  name -> LatLon
                  |
                  v
PlanRunResult(waypoints=[LatLon, ...])  ->  GUI draws it, drones fly it
```

Everything above the `run_enhsp` line is text manipulation: the code never builds a PDDL syntax tree, it edits the flat facts in the problem file as strings. Everything below it is parsing ENHSP's console output back into structured data.

2. domain.pddl explained

`plans/travell/domain.pddl` declares the **vocabulary** and the **physics** of the mission: what can be said about a drone, what numbers it carries, and what actions may change them. It never mentions a specific place - it is reusable for any point-to-point mission anywhere.

2.1 Header and requirements

domain.pddl - lines 1-9

```
(define (domain swarm-drone-mission)

    (:requirements
        :strips
        :typing
        :numeric-fluents
        :negative-preconditions
        :conditional-effects
    )
```

The domain name `swarm-drone-mission` is load-bearing. `services/plan_service.py` reads it out of this file with the `_DOMAIN_NAME` regex and looks it up in the `_DOMAIN_KIND` table to decide which of the four planning paths to take. The **folder** name (`travell`) is only a label in the GUI's combo box; the domain name is what actually selects behaviour.

The requirements list tells ENHSP which PDDL features to expect. `:typing` enables the `?d - drone` style parameter types below; `:numeric-fluents` enables the battery/health/distance arithmetic, which is the reason ENHSP (a **numeric** planner) is needed rather than a classical one; `:negative-preconditions` allows `(not (gps-ok ?d))` to appear in a precondition, which is how the fault-detection actions are written.

2.2 Types

domain.pddl - lines 12-17

```
(:types
    drone
    location
    waypoint
    controller
)
```

Four object classes. `drone` is an aircraft; `location` is any point the drone can be at (the source, the destination, and every intermediate waypoint); `controller` is the ground station. `waypoint` is declared but barely used - note that in problem.pddl the actual waypoints are declared as `location`, not `waypoint`, because the `at`/`connected`/`safe-route` predicates are all typed over `location`. The `waypoint` type survives only for the `(no-fly-zone ?w - waypoint)` predicate, which nothing in the action set reads.

2.3 Predicates - the true/false facts

A predicate is a statement that is either true or false in a given state. ENHSP searches by flipping these on and off. They fall into six groups.

Group A - where things are and what connects to what

```
;; Drone position
(at ?d - drone ?l - location)

;; Mission locations
(source ?l - location)
(destination ?l - location)

;; Route information
(connected ?from - location ?to - location)
(safe-route ?from - location ?to - location)
(restricted-route ?from - location ?to - location)
```

`(at ?d ?l)` is the drone's position. Exactly one is true per drone at a time; the `travel` action adds the new one and deletes the old one in the same effect.

`(connected ?from ?to)` means a leg physically exists in the waypoint graph. `(safe-route ?from ?to)` means that leg is **legal to fly**. The distinction is what makes the no-fly logic work: the retargeting code never deletes `connected` facts, it only rewrites which legs carry `safe-route`. `restricted-route` is documentation - no action reads it.

Group B - flight state and the pre-flight check chain

```
;; Drone state
(flying ?d - drone)
(hovering ?d - drone)
(mission-completed ?d - drone)

;; Mission checks
(route-planned ?d - drone)
(distance-checked ?d - drone)
(battery-checked ?d - drone)
```

The three `-checked`/`-planned` flags force a fixed order on the plan: a drone must have its distance checked before its battery may be checked, its battery checked before a route may be planned, and a route planned before it may travel. This is what makes a Travell plan read as a sensible procedure rather than just 'teleport to the goal'. `route-planned` is deliberately **deleted** again by `travel`, so every single leg has to be re-planned.

Group C - health, and the three ways a drone can fail

```
;; System health
(gps-ok ?d - drone)
(communication-ok ?d - drone)
(drone-healthy ?d - drone)

;; Failure states
(gps-lost ?d - drone)
(communication-lost ?d - drone)
(health-failure ?d - drone)
```

`communication-ok` is the drone's radio link **to the ground controller** - see chapter 5. All three `-ok` flags are preconditions of `plan-safe-route` and `travel`, so a drone that has lost any of them simply cannot be given a route or moved. The matching `-lost`/`-failure` facts are set by the detection actions and are what `notify-ground-controller` triggers on.

Group D - the ground controller's side of the conversation

```
;; Controller commands and status
(controller-notified ?d - drone)
(controller-return ?d - drone)
(controller-hover ?d - drone)
(controller-sacrifice ?d - drone)

;; Emergency states
(emergency-return ?d - drone)
(sacrificed ?d - drone)

;; Warning messages
(battery-insufficient ?d - drone)
(health-warning ?d - drone)
(gps-warning ?d - drone)
(communication-warning ?d - drone)
```

This is the command channel modelled as facts. `controller-notified` is set when the drone has reported a fault upward. `controller-return`, `controller-hover` and `controller-sacrifice` are the three orders that can come back **down**; they are inputs to the problem, not something any action produces, so in the shipped template none of them are ever true and the emergency branches stay dormant. The `-warning` facts are pure telemetry: nothing reads them.

2.4 Functions - the numeric fluents

domain.pddl - lines 78-106

```
(:functions
    (battery ?d - drone)
    (max-battery ?d - drone)
    (energy-required ?from - location ?to - location)
    (distance ?from - location ?to - location)
    (health ?d - drone)
    (minimum-health ?d - drone)
    (minimum-return-battery ?d - drone)
    (hover-energy ?d - drone)
    (route-energy ?d - drone)
    (latitude ?l - location)
    (longitude ?l - location)
)
```

These are real numbers ENHSP compares and updates. The pairing that matters most is `battery` against `energy-required`: `travel` may only fire when `(> (battery ?d) (energy-required ?from ?to))`, and it **decreases** the battery by exactly that amount. A route that costs more than the drone has is therefore genuinely unsolvable, not merely expensive - which is precisely what the 7 'unsolvable' Travell runs on record were.

`latitude` and `longitude` are never read by any action. They exist purely so the Python side can carry geography inside the problem file: `retarget_problem` writes them, `read_locations` reads them back to turn the planner's location **names** into map points. `route-energy` is declared but unused.

2.5 The 16 actions

Each action is a rule: **if** the preconditions hold, ENHSP may apply it, and the effects describe the resulting state. The order of actions in a plan is discovered by search, not written down anywhere.

Phase 1 - pre-flight checks

ACTION 1 of 16 - check-distance

```
(:action check-distance
    :parameters (?d - drone ?from - location ?to - location)
    :precondition (and (at ?d ?from)
                       (connected ?from ?to))
    :effect (distance-checked ?d))
```

The first thing any drone does. It requires only that the drone is at `?from` and that a leg to `?to` exists in the graph, and it sets the `distance-checked` flag that unlocks the battery check. Note what it does **not** do: it never compares the distance against anything. It is a sequencing gate that makes the generated plan mirror a real pre-flight procedure.

ACTION 2 of 16 - check-battery

```
(:action check-battery
    :parameters (?d - drone ?from - location ?to - location)
    :precondition (and (at ?d ?from)
                       (distance-checked ?d)
                       (> (battery ?d) (energy-required ?from ?to)))
    :effect (battery-checked ?d))
```

The first genuinely numeric test. The drone may only certify its battery for a leg it can actually afford. If the leg costs more than the drone holds, this action is inapplicable and `battery-checked` never becomes true, which blocks `plan-safe-route` and therefore `travel` - the mission is then correctly reported unsolvable.

ACTION 3 of 16 - notify-insufficient-battery

```
(:action notify-insufficient-battery
    :parameters (?d - drone ?from - location ?to - location)
    :precondition (and (at ?d ?from)
                       (distance-checked ?d)
                       (<= (battery ?d) (energy-required ?from ?to)))
    :effect (battery-insufficient ?d))
```

The exact mirror image of `check-battery`: same parameters, same first two preconditions, the numeric test inverted from > to <=. It raises a flag recording **why** the mission cannot proceed. Because nothing in the goal or in any other precondition mentions `battery-insufficient`, ENHSP has no reason to ever include it in a successful plan - it is a diagnostic branch, reachable but never useful.

ACTION 4 of 16 - plan-safe-route

```
(:action plan-safe-route
    :parameters (?d - drone ?from - location ?to - location)
    :precondition (and (at ?d ?from)
                       (connected ?from ?to)
                       (safe-route ?from ?to)
                       (battery-checked ?d)
                       (gps-ok ?d)
                       (communication-ok ?d)
                       (drone-healthy ?d))
    :effect (route-planned ?d))
```

The gate that ties everything together, and the place where the no-fly logic actually bites. A leg that is `connected` but has had its `safe-route` fact stripped by the Python retargeting layer cannot be planned here, and so can never be flown.

It also demands the full health triple. This is the point in the plan where the drone confirms it still has GPS, still has a link to the ground controller, and is itself airworthy - before committing to a leg.

Phase 2 - flying the route

ACTION 5 of 16 - travel [the central action]

```
(:action travel
    :parameters (?d - drone ?from - location ?to - location)
    :precondition (and (at ?d ?from)
                       (route-planned ?d)
                       (safe-route ?from ?to)
                       (gps-ok ?d) (communication-ok ?d) (drone-healthy ?d)
                       (> (battery ?d) (energy-required ?from ?to)))
    :effect (and (at ?d ?to)
                 (not (at ?d ?from))
                 (flying ?d)
                 (decrease (battery ?d) (energy-required ?from ?to))
                 (not (route-planned ?d))))
```

This is the only action that moves a drone, and the only one the Python side extracts: `extract_route` in `engine/pddl_planner.py` looks for `travel` steps and nothing else. The route the swarm flies is literally the chain of `?from`/`?to` arguments of the `travel` actions ENHSP chose.

The effect does four things at once: teleports the drone along the leg (add `at ?to`, delete `at ?from`), marks it `flying`, bills the battery for the leg, and - crucially - **clears** `route-planned`. That last deletion forces `plan-safe-route` to run again before the next leg, so a multi-leg detour produces a repeating check/plan/travel rhythm rather than one plan followed by several free moves.

The battery test is repeated here even though `check-battery` already made it. That is deliberate: `check-battery` may have been satisfied for a **different** leg, since the flags are per-drone rather than per-leg.

ACTION 6 of 16 - reach-destination

```
(:action reach-destination
    :parameters (?d - drone ?dest - location)
    :precondition (and (at ?d ?dest)
                       (destination ?dest))
    :effect (and (mission-completed ?d)
                 (not (flying ?d))))
```

Converts arrival into mission success. It is the only producer of `mission-completed`, which the goal demands for every drone, so every successful plan ends with one of these per drone. Clearing `flying` is what stops the fault-detection actions below from firing after landing.

Phase 3 - fault detection

All three detectors share a shape: they require `(flying ?d)` - a fault is only interesting in the air - they set a failure fact plus a warning fact, and they clear `flying`, which effectively grounds the drone and makes `notify-ground-controller` applicable.

ACTION 7 of 16 - detect-gps-loss

```
(:action detect-gps-loss
    :parameters (?d - drone)
    :precondition (and (flying ?d)
                       (not (gps-ok ?d)))
    :effect (and (gps-lost ?d) (gps-warning ?d) (not (flying ?d))))
```

Fires when a drone is airborne without a valid GPS fix. Because `problem.pddl` asserts `(gps-ok drone1)` in the initial state and no action ever deletes it, this can never fire in the shipped template. It is there so that a problem file describing a **degraded** drone - one where `gps-ok` is simply omitted - plans a sensible response instead of being unsolvable.

ACTION 8 of 16 - detect-communication-loss

```
(:action detect-communication-loss
    :parameters (?d - drone)
    :precondition (and (flying ?d)
                       (not (communication-ok ?d)))
    :effect (and (communication-lost ?d)
                 (communication-warning ?d)
                 (not (flying ?d))))
```

The communications counterpart, and the one that matters for chapter 5. It models the drone noticing that its link to the ground controller has dropped. In Travell there is no notion of drone-to-drone links at all - that arrives only in the two formation plans - so this single flag is the whole communications model.

ACTION 9 of 16 - detect-health-failure

```
(:action detect-health-failure
    :parameters (?d - drone)
    :precondition (and (flying ?d)
                       (< (health ?d) (minimum-health ?d)))
    :effect (and (health-failure ?d) (health-warning ?d) (not (flying ?d))))
```

The numeric detector: it compares the drone's `health` against its own `minimum-health` threshold rather than testing a flag. With the template's 95 against 40 it cannot fire, but a problem file that starts a drone below its threshold will plan around it.

Phase 4 - the controller loop

ACTION 10 of 16 - notify-ground-controller

```
(:action notify-ground-controller
    :parameters (?d - drone ?c - controller)
    :precondition (or (gps-lost ?d)
                      (communication-lost ?d)
                      (health-failure ?d))
    :effect (controller-notified ?d))
```

The single upward message in the whole domain. Any one of the three faults makes it applicable - this is the only action in the Travell domain with a disjunctive precondition.

There is a deliberate irony worth noting for the write-up: a drone that has lost communications can still execute `notify-ground-controller`, because the action does not require `communication-ok`. The model treats notification as best-effort rather than as something the lost link would prevent.

ACTIONS 11 and 13 of 16 - the two controller orders

```
(:action controller-order-return
    :parameters (?d - drone)
    :precondition (and (controller-notified ?d) (controller-return ?d))
    :effect (emergency-return ?d))

(:action controller-order-hover
    :parameters (?d - drone)
    :precondition (and (controller-notified ?d) (controller-hover ?d))
    :effect (hovering ?d))
```

Each turns a standing order into an active state, but only after the drone has reported in. `controller-return` and `controller-hover` are never produced by any action, so they have to be present in the initial state for these to fire - that is how a problem author pre-declares 'if this drone gets into trouble, bring it home' versus 'make it hold'.

ACTION 12 of 16 - return-to-source

```
(:action return-to-source
    :parameters (?d - drone ?current - location ?src - location)
    :precondition (and (at ?d ?current)
                       (source ?src)
                       (emergency-return ?d)
                       (safe-route ?current ?src)
                       (> (battery ?d) (energy-required ?current ?src)))
    :effect (and (at ?d ?src) (not (at ?d ?current))
                 (not (emergency-return ?d))
                 (decrease (battery ?d) (energy-required ?current ?src))))
```

The abort manoeuvre: fly straight back to the source, paying the battery for it, and clear the emergency flag on arrival. It still insists on a `safe-route`, so an emergency return also respects the no-fly zone. Note it requires a **direct** safe leg from wherever the drone is back to the source, which on a multi-leg detour may not exist - a real limitation of the model.

ACTIONS 14 and 15 of 16 - the hover loop and its exit

```
(:action hover
    :parameters (?d - drone)
    :precondition (and (hovering ?d)
                       (> (battery ?d) (minimum-return-battery ?d)))
    :effect (decrease (battery ?d) (hover-energy ?d)))

(:action initiate-low-battery-return
    :parameters (?d - drone)
    :precondition (and (hovering ?d)
                       (<= (battery ?d) (minimum-return-battery ?d)))
    :effect (and (emergency-return ?d) (not (hovering ?d))))
```

These two form a numeric loop and are the most interesting pair in the domain. `hover` can be applied repeatedly, each time draining `hover-energy` (2 units in the template) from the battery. It stays applicable only while the battery is above the return reserve. The moment it drops to or below that reserve, `hover` becomes inapplicable and `initiate-low-battery-return` becomes applicable instead - the drone stops holding and heads home on its remaining margin. Together they encode 'hold until your reserve is gone, then come back'.

ACTION 16 of 16 - sacrifice-drone

```
(:action sacrifice-drone
    :parameters (?d - drone)
    :precondition (and (controller-notified ?d) (controller-sacrifice ?d))
    :effect (and (sacrificed ?d) (not (flying ?d)) (not (hovering ?d))))
```

The last-resort order: give the drone up where it is. It clears both airborne states and marks it `sacrificed`. Like the other two orders it needs a pre-declared `controller-sacrifice` fact, so it is dormant in the shipped template.

3. problem.pddl explained

Where the domain says what is **possible**, `plans/travell/problem.pddl` says what is **true right now** and what counts as success. Remember that this file is a template: the coordinates and distances below are the Chennai scenario it was authored against, and every one of them is overwritten before ENHSP ever sees it.

problem.pddl - the objects

```
(define (problem drone-mission-001)
    (:domain swarm-drone-mission)

    (:objects
        drone1 - drone
        ground-controller - controller
        source - location
        destination - location
        waypoint1 - location
        waypoint2 - location
    ))
```

Six objects. Note that `waypoint1` and `waypoint2` are declared as type `location`, not type `waypoint` - as discussed in section 2.2, everything that can be flown to must be a `location` because that is what `at`, `connected` and `safe-route` are typed over.

There is exactly **one** drone here. This is the reference drone the template is written around; `expand_point_to_point_drones` clones it into `drone1 .. droneN` at run time. The constant `PLAN_DRONE_OBJECT = "drone1"` in `plan_service.py` is the name the code later looks for when extracting the route.

problem.pddl - the waypoint graph

```
;; Locations connected by possible routes
(connected source destination)

(connected source waypoint1)
(connected waypoint1 waypoint2)
(connected waypoint2 destination)

(connected waypoint1 source)
(connected waypoint2 waypoint1)
(connected destination waypoint2)

;; Safe routes avoid the restricted direct route
(safe-route source waypoint1)
(safe-route waypoint1 waypoint2)
(safe-route waypoint2 destination)
... (and the three reverse legs)

;; Direct route is restricted
(restricted-route source destination)
```

This is the shape the whole template exists to encode: a **direct** leg source-to-destination, and a **detour** of three legs via two waypoints that swings around it. Every leg is declared in both directions.

Critically, the direct leg is `connected` but is **not** `safe-route`. As written, the template therefore always detours. At run time the Python layer overwrites this decision entirely: `_set_direct_route_only` strips every `safe-route` and adds back only the direct pair when nothing blocks the line, while `_set_detour_route` strips them and builds a fresh chain through computed via-points when something does.

problem.pddl - geography and cost

```
;; Geographic coordinates
(= (latitude source) 13.0827)       (= (longitude source) 80.2707)
(= (latitude destination) 13.0674)  (= (longitude destination) 80.2376)
(= (latitude waypoint1) 13.0830)    (= (longitude waypoint1) 80.2250)
(= (latitude waypoint2) 13.0650)    (= (longitude waypoint2) 80.2250)

;; Distances are provided by the mission-generation program
(= (distance source destination) 4300)
(= (distance source waypoint1) 3900)
...
;; Energy required for each route
(= (energy-required source destination) 45)
(= (energy-required source waypoint1) 20)
(= (energy-required waypoint1 waypoint2) 12)
(= (energy-required waypoint2 destination) 10)
```

Four points near Chennai, with a hand-authored distance and energy cost per leg. The comment 'provided by the mission-generation program' is exactly right: `retarget_problem` recomputes every one of these numbers from the moved coordinates.

The per-leg costs are not uniform, and that is intentional. The direct leg costs 45 units over 4300 m (about 0.0105 per metre) while the detour legs cost 20/3900, 12/2100 and 10/1800 (about 0.0051 to 0.0057 per metre) - the restricted direct route is roughly twice as expensive per metre. `retarget_problem` preserves each leg's **own** rate when it recomputes, rather than applying one global rate, precisely so this authored asymmetry survives being moved across the map.

problem.pddl - the drone's initial state

```
;; Drone starts in a normal operating state
(at drone1 source)
(gps-ok drone1) (communication-ok drone1) (drone-healthy drone1)

(= (battery drone1) 85)            (= (max-battery drone1) 100)
(= (minimum-return-battery drone1) 30)
(= (hover-energy drone1) 2)
(= (health drone1) 95)             (= (minimum-health drone1) 40)

;; Waypoint 1 is inside the restricted area
(no-fly-zone waypoint1)
```

A healthy drone at the source with 85 of 100 battery and 95 health against a floor of 40. These numbers are what make the three fault detectors dormant: health is comfortably above its minimum and all three `-ok` flags are set.

The 85 battery is worth checking against the route costs. The detour costs 20 + 12 + 10 = 42 units, leaving 43. The direct leg costs 45, leaving 40. Both are affordable, so battery is not what forces the detour - the missing `safe-route` fact is.

`(no-fly-zone waypoint1)` is a leftover. No action reads it, and the retargeting code explicitly refuses to infer a restriction from it - as `retarget_problem`'s docstring explains, that point moves proportionally with the rest of the graph, so whether it 'looked blocked' would depend on how far apart the operator's two clicks were rather than on any real obstacle.

problem.pddl - the goal

```
(:goal
    (and
        (mission-completed drone1)
        (at drone1 destination)))
```

Two conjuncts per drone. `at drone1 destination` alone would be satisfiable by `travel` without ever running `reach-destination`; demanding `mission-completed` as well forces that closing action into every plan. `expand_point_to_point_drones` clones both lines for every extra drone, which is why an 8-drone run's goal carries 16 conjuncts.

4. Every Python function in the Travell path

Three modules do the work: `services/plan_service.py` orchestrates, `engine/pddl_problem.py` rewrites the problem file, and `engine/pddl_planner.py` runs ENHSP and parses its output. They are presented here in the order a run touches them.

4.1 services/plan_service.py - orchestration

_find_problem_file(plan_dir)

```
def _find_problem_file(plan_dir: Path) -> Optional[Path]:
    exact = plan_dir / "problem.pddl"
    if exact.is_file():
        return exact
    candidates = [p for p in plan_dir.glob("*.pddl")
                  if p.name.lower().endswith("problem.pddl")]
    return candidates[0] if len(candidates) == 1 else None
```

Locates a plan folder's problem file. It prefers the exact name `problem.pddl`, which is what Travell uses, but falls back to any single file ending in `problem.pddl` - that fallback exists for the Search plan, whose file is named `SearchProblem.pddl`. Returning `None` when two candidates match is a safety measure: an ambiguous folder is rejected rather than guessed at.

list_plans(plans_dir)

```
def list_plans(plans_dir: Path = PLANS_DIR) -> list[str]:
    if not plans_dir.is_dir():
        return []
    return sorted(
        p.name for p in plans_dir.iterdir()
        if p.is_dir() and (p / "domain.pddl").is_file()
        and _find_problem_file(p) is not None)
```

Fills the GUI's 'Named Plan' combo box. A folder only counts as a plan if it has both a domain and a findable problem file, so a half-finished folder never appears in the UI. This is why all four plan folders show up but the `runs/` sub-folders do not.

plan_kind(plan_name) and the _DOMAIN_KIND table

```
_DOMAIN_KIND = {
    "swarm-drone-mission":          "point_to_point",
    "forest-drone-search":          "area_coverage",
    "v-formation-drone-mission":    "formation",
    "grid-formation-drone-mission": "grid_formation",
}
_DOMAIN_NAME = re.compile(r"\(define\s*\(domain\s+(\S+)\)")

def plan_kind(plan_name: str, plans_dir: Path = PLANS_DIR) -> str:
    domain_path = plans_dir / plan_name / "domain.pddl"
    try:
        text = domain_path.read_text(encoding="utf-8")
    except OSError:
        return "point_to_point"
    match = _DOMAIN_NAME.search(text)
    return _DOMAIN_KIND.get(match.group(1), "point_to_point") if match else "point_to_point"
```

The dispatcher for the whole system. It opens the folder's `domain.pddl`, pulls the domain name out of the `(define (domain ...))` line, and maps it to one of four behaviours. Travell's `swarm-drone-mission` maps to `point_to_point`.

Every failure path defaults to `point_to_point` - a missing file, an unreadable file, an unrecognised domain name. That default is the behaviour every plan had before the other three kinds existed, so an unknown plan degrades to the oldest, simplest path rather than erroring.

Because the decision is made from the **domain name** and not the folder name, a new folder that reuses `swarm-drone-mission` works immediately with no code change.

class PlanRunResult

```
@dataclass
class PlanRunResult:
    plan_name: str
    steps: list[PlanStep]
    waypoints: list[LatLon]
    location_names: list[str]
    run_dir: Path
    per_drone_waypoints: Optional[dict[str, list[LatLon]]] = None
```

The single object handed back to the GUI. For Travell, `waypoints` is the one shared route the whole swarm flies and `per_drone_waypoints` stays `None` - that field is only populated by the three plans where each drone has its own distinct track. `steps` is the raw ENHSP action list, used by the GUI to show what the planner decided; `run_dir` points at the timestamped folder holding the concrete problem and the planner log.

_Worker.run(...) - the thread-side entry point

```
class _Worker(QObject):
    finished = Signal(object)   # PlanRunResult
    failed = Signal(str)

    @Slot(str, object, object, object, int)
    def run(self, plan_name, source, destination, no_fly_zone, drone_count=1):
        try:
            result = self._run(plan_name, source, destination, no_fly_zone, drone_count)
        except PlanNotFound as exc:
            self._save_stdout(plan_name, exc.stdout)
            self.failed.emit(str(exc))
        except PlannerError as exc:
            self.failed.emit(str(exc))
        except OSError as exc:
            self.failed.emit(f"Could not read/write plan files: {exc}")
        except ValueError as exc:
            self.failed.emit(str(exc))
        else:
            self.finished.emit(result)
```

Runs on a background `QThread`, because `run_enhsp` shells out to Java and blocks for seconds. Doing it on the GUI thread would freeze the map.

The value of this wrapper is that it converts **every** failure mode into a `failed` signal carrying a readable string, so a genuinely unsolvable mission, a missing Java install and an unwritable folder all surface as a message in the UI rather than a traceback. The `PlanNotFound` branch additionally calls `_save_stdout` first, so ENHSP's reasoning is preserved on disk even for a run that produced no plan - which is how the four failed Grid-Formation runs were later diagnosed.

_save_stdout, _plan_files and _start_run - the bookkeeping trio

```
def _save_stdout(self, plan_name: str, stdout: str) -> None:
    run_dir = getattr(self, "_last_run_dir", None)
    if run_dir is None or not stdout:
        return
    try:
        (run_dir / "plan.txt").write_text(stdout, encoding="utf-8")
    except OSError:
        pass

def _plan_files(self, plan_name: str) -> tuple[Path, Path]:
    plan_dir = PLANS_DIR / plan_name
    domain_path = plan_dir / "domain.pddl"
    template_path = _find_problem_file(plan_dir)
    if not domain_path.is_file() or template_path is None:
        raise PlannerError(f"Plan '{plan_name}' is missing domain.pddl/problem.pddl")
    return domain_path, template_path

def _start_run(self, plan_name: str, concrete_text: str) -> Path:
    run_dir = PLANS_DIR / plan_name / "runs" / time.strftime("%Y-%m-%d_%H%M%S")
    run_dir.mkdir(parents=True, exist_ok=True)
    self._last_run_dir = run_dir
    (run_dir / "problem.pddl").write_text(concrete_text, encoding="utf-8")
    return run_dir
```

`_plan_files` resolves and validates the two template paths up front, so a broken plan folder fails with a clear message before any work is done.

`_start_run` creates the timestamped `runs/<date>_<time>/` folder, records it on the worker as `_last_run_dir`, and writes the concrete problem into it. This is why every run leaves behind the **exact** problem ENHSP was given - the audit trail behind the run counts in `plans/PLANS_STATUS.txt`.

`_save_stdout` is best-effort by design: it is called on the failure path, so it swallows its own `OSError` rather than masking the original planning failure with a disk error.

_Worker._run(...) - the Travell run, start to finish

```
def _run(self, plan_name, source, destination, no_fly_zone, drone_count=1):
    if plan_kind(plan_name) == "formation":
        return self._run_formation(...)
    if plan_kind(plan_name) == "grid_formation":
        return self._run_grid_formation(...)

    domain_path, template_path = self._plan_files(plan_name)
    template_text = template_path.read_text(encoding="utf-8")
    concrete_text = retarget_problem(
        template_text,
        PddlLatLon(lat=source.lat, lon=source.lon),
        PddlLatLon(lat=destination.lat, lon=destination.lon),
        no_fly_zone=([PddlLatLon(lat=p.lat, lon=p.lon) for p in no_fly_zone]
                     if no_fly_zone else None))
    concrete_text = expand_point_to_point_drones(concrete_text, max(1, drone_count))

    run_dir = self._start_run(plan_name, concrete_text)
    steps, stdout = run_enhsp(domain_path, run_dir / "problem.pddl")
    (run_dir / "plan.txt").write_text(stdout, encoding="utf-8")

    location_names = extract_route(steps, PLAN_DRONE_OBJECT)
    if not location_names:
        raise PlanNotFound(
            f"ENHSP found a plan but it contains no '{PLAN_DRONE_OBJECT}' travel legs.")
    locations = read_locations(concrete_text)
    try:
        waypoints = [LatLon(lat=locations[name].lat, lon=locations[name].lon)
                     for name in location_names]
    except KeyError as exc:
        raise PlanNotFound(
            f"Plan references location {exc} with no coordinates in problem.pddl") from exc

    return PlanRunResult(plan_name=plan_name, steps=steps, waypoints=waypoints,
                         location_names=location_names, run_dir=run_dir)
```

This is the function that **is** the Travell mission. The two early returns at the top are the dispatch to the other two formation kinds - the Search plan arrives through a separate slot entirely, because it is picked with different clicks.

The body is a strict six-step sequence: read the template, retarget it onto the picked points, expand it to N drones, persist it, solve it, and read the answer back. Note the **order** of retarget-then-expand: retargeting moves a single-drone graph, which is cheap, and only then is the drone cloned. Doing it the other way round would rewrite every drone's coordinates N times over.

The two `PlanNotFound` raises guard the join between the planner's world and the map's. The first catches a plan that solved the goal without any `travel` legs; the second catches a plan that names a location the problem file has no coordinates for. Both would otherwise surface much later as an empty or broken flight path.

class PlanService - the GUI-facing handle

```
class PlanService(QObject):
    finished = Signal(object)
    failed = Signal(str)
    _run_requested = Signal(str, object, object, object, int)

    def __init__(self, parent=None):
        super().__init__(parent)
        self._thread = QThread(self)
        self._worker = _Worker()
        self._worker.moveToThread(self._thread)
        self._worker.finished.connect(self.finished)
        self._worker.failed.connect(self.failed)
        self._run_requested.connect(self._worker.run)
        self._thread.start()

    def run_async(self, plan_name, source, destination, no_fly_zone=None, drone_count=1):
        self._run_requested.emit(plan_name, source, destination, no_fly_zone, drone_count)

    def shutdown(self) -> None:
        self._thread.quit()
        self._thread.wait(2000)
```

The standard Qt worker-object pattern. `PlanService` lives on the GUI thread; `_Worker` is moved onto a private `QThread`. The call crosses threads through the `_run_requested` signal rather than by calling the worker directly, which is what makes the hand-off thread-safe: Qt queues the invocation and runs it on the worker's thread.

`run_async` returns immediately. The GUI learns the outcome later through `finished` or `failed`, which `PlanService` simply re-emits from the worker's own signals.

`shutdown` asks the thread to quit and waits up to two seconds, so closing the application mid-plan does not leave an orphaned Java subprocess owner behind.

4.2 engine/pddl_problem.py - rewriting the problem file

This module's whole job is: given the template text and two new points, return new template text. It never parses PDDL - it edits the flat facts as strings with regular expressions.

Geometry primitives

haversine_m(a, b)

```
EARTH_RADIUS_M = 6_371_000.0
METRES_PER_DEG_LAT = 111_320.0

def haversine_m(a: LatLon, b: LatLon) -> float:
    lat1, lat2 = math.radians(a.lat), math.radians(b.lat)
    dlat = lat2 - lat1
    dlon = math.radians(b.lon - a.lon)
    h = (math.sin(dlat / 2) ** 2
         + math.cos(lat1) * math.cos(lat2) * math.sin(dlon / 2) ** 2)
    return 2 * EARTH_RADIUS_M * math.asin(min(1.0, math.sqrt(h)))
```

Great-circle distance in metres between two latitude/longitude points. This is the function that produces every `distance` fact written into a concrete problem file. The `min(1.0, ...)` clamp guards against a floating-point value slightly above 1 reaching `asin`, which would raise a domain error for two points that are effectively identical.

_metres_per_deg_lon, _to_local and _from_local - the local flat frame

```
def _metres_per_deg_lon(lat_deg: float) -> float:
    return METRES_PER_DEG_LAT * math.cos(math.radians(lat_deg))

def _to_local(point: LatLon, origin: LatLon) -> tuple[float, float]:
    x = (point.lon - origin.lon) * _metres_per_deg_lon(origin.lat)
    y = (point.lat - origin.lat) * METRES_PER_DEG_LAT
    return x, y

def _from_local(offset: tuple[float, float], origin: LatLon) -> LatLon:
    x, y = offset
    return LatLon(lat=origin.lat + y / METRES_PER_DEG_LAT,
                  lon=origin.lon + x / _metres_per_deg_lon(origin.lat))
```

The three functions that let the rest of the module do ordinary plane geometry on geographic points. `_to_local` converts a lat/lon into metres east and metres north of a chosen origin; `_from_local` converts back.

This is an equirectangular projection centred on the origin. A degree of latitude is a fixed 111320 m everywhere, but a degree of longitude shrinks towards the poles by `cos(latitude)` - which is exactly what `_metres_per_deg_lon` applies. The approximation is accurate at the few-kilometre scale these missions operate at, and it is what makes offsets like 'ten metres to the left' expressible at all.

_similarity_transform(...) - the heart of retargeting

```
def _similarity_transform(old_source, old_dest, new_source, new_dest):
    old_vec = _to_local(old_dest, old_source)
    old_len = math.hypot(*old_vec)
    if old_len < 1.0:
        def translate_only(point: LatLon) -> LatLon:
            return _from_local(_to_local(point, old_source), new_source)
        return translate_only

    new_vec = _to_local(new_dest, new_source)
    scale = math.hypot(*new_vec) / old_len
    angle = math.atan2(new_vec[1], new_vec[0]) - math.atan2(old_vec[1], old_vec[0])
    cos_a, sin_a = math.cos(angle), math.sin(angle)

    def transform(point: LatLon) -> LatLon:
        x, y = _to_local(point, old_source)
        rotated = (x * cos_a - y * sin_a, x * sin_a + y * cos_a)
        scaled = (rotated[0] * scale, rotated[1] * scale)
        return _from_local(scaled, new_source)
    return transform
```

Returns a **function** that maps any point through the rotate-plus-scale-plus-translate which carries the template's source onto the new source and the template's destination onto the new destination.

The reason this exists rather than simply moving the two endpoints: a waypoint graph is a **shape**. The template's detour swings a certain distance out to a certain side of the direct line. If only the endpoints moved, the waypoints would stay near Chennai while the mission moved to Warangal, and the 'detour' would be nonsense. By carrying every location through the same transform, the detour's proportions are preserved wherever the mission is retargeted.

The `old_len < 1.0` guard handles a degenerate template whose source and destination coincide: there is no direction to rotate onto and no length to scale by, so it falls back to a pure translation.

The transform is built once and applied to every location, which is what keeps the graph internally consistent.

Reading and writing facts

read_locations(problem_text)

```
_LATLON_FACT = re.compile(
    r"\(=\s*\((latitude|longitude)\s+(\S+)\)\s*([-\d.eE]+)\s*\)")

def read_locations(problem_text: str) -> dict[str, LatLon]:
    lats: dict[str, float] = {}
    lons: dict[str, float] = {}
    for kind, name, value in _LATLON_FACT.findall(problem_text):
        target = lats if kind == "latitude" else lons
        target[name] = float(value)
    return {name: LatLon(lat=lats[name], lon=lons[name])
            for name in lats.keys() & lons.keys()}
```

Scans a problem file for every latitude and longitude fact and merges them by object name. The set intersection `lats.keys() & lons.keys()` is a quiet correctness detail: a location with a latitude but no longitude is dropped rather than raising, so a partially written file degrades instead of crashing.

This function is used twice per run, at both ends of the pipeline - once by `retarget_problem` to learn where the template's locations currently are, and once by `_run` afterwards to turn the planner's location **names** back into map coordinates.

_fact_value, _set_fact_value and _set_latlon - the two-argument fact accessors

```
def _fact_value(problem_text, predicate, a, b) -> float | None:
    pattern = re.compile(
        rf"\(=\s*\({re.escape(predicate)}\s+{re.escape(a)}\s+{re.escape(b)}\)"
        rf"\s*([-\d.eE]+)\s*\)")
    match = pattern.search(problem_text)
    return float(match.group(1)) if match else None

def _set_fact_value(problem_text, predicate, a, b, value) -> str:
    pattern = re.compile(
        rf"(\(=\s*\({re.escape(predicate)}\s+{re.escape(a)}\s+{re.escape(b)}\)\s*)"
        rf"[-\d.eE]+(\s*\))")
    def _repl(match): return f"{match.group(1)}{value:.2f}{match.group(2)}"
    return pattern.sub(_repl, problem_text, count=1)

def _set_latlon(problem_text: str, name: str, point: LatLon) -> str:
    for kind, value in (("latitude", point.lat), ("longitude", point.lon)):
        pattern = re.compile(
            rf"(\(=\s*\({kind}\s+{re.escape(name)}\)\s*)[-\d.eE]+(\s*\))")
        problem_text = pattern.sub(
            lambda m, v=value: f"{m.group(1)}{v:.7f}{m.group(2)}", problem_text, count=1)
    return problem_text
```

The read/write pair for binary numeric facts such as `(= (distance source destination) 4300)`. `_fact_value` returns `None` rather than raising when the fact is absent, which callers use to mean 'this edge had no authored cost'.

`_set_fact_value` rewrites **only the number**, keeping the surrounding text and the original whitespace via the two captured groups. `count=1` ensures a single substitution.

`_set_latlon` is the one-argument equivalent for coordinates, and writes 7 decimal places against `_set_fact_value`'s 2 - roughly a centimetre of precision, appropriate for coordinates, where two decimals on a distance in metres is ample.

_find_matching_paren and _section_span - locating a PDDL block

```
def _find_matching_paren(text: str, open_pos: int) -> int:
    depth = 0
    for i in range(open_pos, len(text)):
        if text[i] == "(": depth += 1
        elif text[i] == ")":
            depth -= 1
            if depth == 0: return i
    raise ValueError("unbalanced parentheses")

def _section_span(text: str, keyword: str) -> tuple[int, int]:
    match = re.search(rf"\(:{re.escape(keyword)}\b", text)
    if not match:
        raise ValueError(f"no (:{keyword} ...) section found")
    open_pos = match.start()
    return open_pos, _find_matching_paren(text, open_pos)
```

The closest this module comes to parsing. `_section_span` finds where a block such as `(:init ...)` or `(:goal ...)` starts and, by counting parenthesis depth, where it ends. Every insertion in the module is made just before that closing index, which is how new facts land **inside** the right block. The `ValueError` on unbalanced parentheses is the failure that would surface if a malformed file were fed in - and is the class of bug the Grid-Formation `hover-altitude` defect belonged to.

_add_locations and _add_init_facts - the two injectors

```
def _add_locations(problem_text: str, names: list[str]) -> str:
    _, close_pos = _section_span(problem_text, "objects")
    addition = "".join(f"\n        {name} - location" for name in names)
    return problem_text[:close_pos] + addition + "\n    " + problem_text[close_pos:]

def _add_init_facts(problem_text: str, facts_text: str) -> str:
    _, close_pos = _section_span(problem_text, "init")
    return problem_text[:close_pos] + facts_text + "\n    " + problem_text[close_pos:]
```

Declare new `location` objects, and append raw fact text into the initial state. Both are pure string splices at the index `_section_span` reported. These two functions are also imported by `engine/search_problem.py`, which builds an entire waypoint graph out of them.

The no-fly geometry

_convex_hull(points) - Andrew's monotone chain

```
def _convex_hull(points: list[_Point]) -> list[_Point]:
    pts = sorted(set(points))
    if len(pts) <= 2: return pts
    def cross(o, a, b):
        return (a[0]-o[0])*(b[1]-o[1]) - (a[1]-o[1])*(b[0]-o[0])
    lower = []
    for p in pts:
        while len(lower) >= 2 and cross(lower[-2], lower[-1], p) <= 0: lower.pop()
        lower.append(p)
    upper = []
    for p in reversed(pts):
        while len(upper) >= 2 and cross(upper[-2], upper[-1], p) <= 0: upper.pop()
        upper.append(p)
    return lower[:-1] + upper[:-1]
```

Computes the convex hull: the tightest simple polygon containing all the given points, counter-clockwise, with duplicates removed.

It exists because of a real usability problem. An operator marking a rectangular no-fly area clicks the corners in whatever order feels natural - very often top-left, top-right, bottom-left, bottom-right. Taken literally as a polygon, that click order is a self-intersecting **bowtie**, not a rectangle, and every geometry test below assumes a simple polygon. Replacing the raw click order with the hull yields the one unambiguous simple polygon those four points define.

_offset_polygon(polygon, distance_m)

```
def _offset_polygon(polygon: list[_Point], distance_m: float) -> list[_Point]:
    n = len(polygon)
    if n < 3 or distance_m <= 0: return list(polygon)
    centroid = (sum(p[0] for p in polygon)/n, sum(p[1] for p in polygon)/n)
    shifted_edges = []
    for i in range(n):
        a, b = polygon[i], polygon[(i+1) % n]
        direction = (b[0]-a[0], b[1]-a[1])
        length = math.hypot(*direction)
        if length < 1e-9: continue
        normal = (direction[1]/length, -direction[0]/length)
        mid = ((a[0]+b[0])/2.0, (a[1]+b[1])/2.0)
        to_centroid = (centroid[0]-mid[0], centroid[1]-mid[1])
        if normal[0]*to_centroid[0] + normal[1]*to_centroid[1] > 0:
            normal = (-normal[0], -normal[1])      # was pointing inward - flip it
        shifted_edges.append(((a[0]+normal[0]*distance_m,
                               a[1]+normal[1]*distance_m), direction))
    ...
    return [_line_intersection(*shifted_edges[i-1], *shifted_edges[i]) for i in range(m)]
```

Grows a convex polygon outward by a fixed margin, so a route planned against the result clears the real boundary by that margin instead of grazing its corners. Travell uses `NO_FLY_BUFFER_M = 10.0`.

The construction is edge-offset-and-reintersect: each edge's supporting line is pushed out along its own outward normal, and the new corners are where consecutive offset lines meet. The dot-product test against the direction to the centroid is what determines which of the two normals is the outward one.

The docstring notes why this is done rather than the simpler alternative of scaling the polygon away from its centroid: that alternative over-expands the corners of an elongated shape, whereas edge offsetting is exact for a convex polygon.

_orientation, _on_segment and _segments_intersect

```
def _orientation(a, b, c) -> float:
    return (b[0]-a[0])*(c[1]-a[1]) - (b[1]-a[1])*(c[0]-a[0])

def _on_segment(a, b, p) -> bool:
    return (min(a[0],b[0])-1e-9 <= p[0] <= max(a[0],b[0])+1e-9 and
            min(a[1],b[1])-1e-9 <= p[1] <= max(a[1],b[1])+1e-9)

def _segments_intersect(p1, p2, p3, p4) -> bool:
    d1, d2 = _orientation(p3,p4,p1), _orientation(p3,p4,p2)
    d3, d4 = _orientation(p1,p2,p3), _orientation(p1,p2,p4)
    if ((d1>0) != (d2>0)) and ((d3>0) != (d4>0)) and d1 and d2 and d3 and d4:
        return True
    if d1 == 0 and _on_segment(p3,p4,p1): return True
    if d2 == 0 and _on_segment(p3,p4,p2): return True
    if d3 == 0 and _on_segment(p1,p2,p3): return True
    if d4 == 0 and _on_segment(p1,p2,p4): return True
    return False
```

The standard computational-geometry trio. `_orientation` returns a signed area whose sign says whether three points turn left, turn right, or are collinear.

Two segments properly cross when each straddles the other's line - that is the first condition, four orientations with opposite signs in both pairs. The four collinear cases that follow handle touching: a zero orientation means a point lies on the other segment's infinite **line**, so `_on_segment` is needed to confirm it lies within the segment's actual span. Without those four checks, an endpoint resting exactly on an edge would be missed.

_point_in_polygon and _segment_crosses_polygon

```
def _point_in_polygon(point: _Point, polygon: list[_Point]) -> bool:
    x, y = point
    inside = False
    x2, y2 = polygon[-1]
    for x1, y1 in polygon:
        if (y1 > y) != (y2 > y):
            x_at_y = x1 + (y - y1) * (x2 - x1) / (y2 - y1)
            if x < x_at_y: inside = not inside
        x2, y2 = x1, y1
    return inside

def _segment_crosses_polygon(seg_a, seg_b, polygon) -> bool:
    if len(polygon) < 3: return False
    n = len(polygon)
    for i in range(n):
        if _segments_intersect(seg_a, seg_b, polygon[i], polygon[(i+1) % n]):
            return True
    midpoint = ((seg_a[0]+seg_b[0])/2.0, (seg_a[1]+seg_b[1])/2.0)
    return _point_in_polygon(midpoint, polygon)
```

`_point_in_polygon` is ray casting: count how many polygon edges a ray from the point crosses; an odd count means inside. The `(y1 > y) != (y2 > y)` test picks exactly the edges that span the point's height.

`_segment_crosses_polygon` is the actual **is this route blocked** test. It is deliberately two-part: a leg is blocked if it crosses any edge, **or** if it lies entirely inside the polygon - the second case crosses no edge at all, so the midpoint test is needed to catch it. This is the predicate `retarget_problem` calls to decide between a direct route and a detour.

_segment_blocked_by_polygon(a, b, polygon)

```
def _points_close(p1, p2, eps: float = 0.5) -> bool:
    return abs(p1[0]-p2[0]) < eps and abs(p1[1]-p2[1]) < eps

def _segment_blocked_by_polygon(a, b, polygon) -> bool:
    n = len(polygon)
    a_idx = next((i for i, p in enumerate(polygon) if _points_close(a, p)), None)
    b_idx = next((i for i, p in enumerate(polygon) if _points_close(b, p)), None)
    if a_idx is not None and b_idx is not None:
        diff = abs(a_idx - b_idx)
        return diff != 1 and diff != n - 1
    for i in range(n):
        p, q = polygon[i], polygon[(i+1) % n]
        a_shared = _points_close(a, p) or _points_close(a, q)
        b_shared = _points_close(b, p) or _points_close(b, q)
        if a_shared and b_shared: continue
        if _segments_intersect(a, b, p, q) and not (a_shared or b_shared): return True
    midpoint = ((a[0]+b[0])/2.0, (a[1]+b[1])/2.0)
    return _point_in_polygon(midpoint, polygon)
```

A more forgiving blocking test than `_segment_crosses_polygon`, written specifically for the visibility graph below, where the candidate endpoints usually **are** polygon corners. Merely touching a corner is not an obstruction; cutting through the interior is.

The first branch is the important one. When both endpoints are polygon vertices, adjacency decides it outright: vertices one apart in the ring - including the wrap-around pair, hence `diff == n - 1` - form a real boundary edge and are fine, while any other pair is a diagonal, which for a convex polygon necessarily crosses the interior.

That branch exists because the general fallback would give the wrong answer here. Its last resort is 'is the midpoint inside?', and an edge's own midpoint sits exactly **on** the boundary - the one case where inside-or-outside is ill-posed. Deciding by exact adjacency sidesteps that entirely.

_shortest_detour(source, destination, polygon)

```
def _shortest_detour(source, destination, polygon) -> list[_Point]:
    nodes = [source, destination] + list(polygon)
    n = len(nodes)
    edge_len = [[math.inf] * n for _ in range(n)]
    for i in range(n):
        edge_len[i][i] = 0.0
        for j in range(i + 1, n):
            if not _segment_blocked_by_polygon(nodes[i], nodes[j], polygon):
                d = math.hypot(nodes[i][0]-nodes[j][0], nodes[i][1]-nodes[j][1])
                edge_len[i][j] = edge_len[j][i] = d
    # Dijkstra from node 0 (source) to node 1 (destination)
    best = [math.inf] * n; best[0] = 0.0
    prev = [-1] * n; visited = [False] * n
    for _ in range(n):
        u = min((i for i in range(n) if not visited[i]),
                key=lambda i: best[i], default=None)
        if u is None or best[u] == math.inf: break
        visited[u] = True
        for v in range(n):
            if not visited[v] and edge_len[u][v] < math.inf:
                candidate = best[u] + edge_len[u][v]
                if candidate < best[v]: best[v] = candidate; prev[v] = u
    if best[1] == math.inf: return [source, destination]
    path_idx = []; cur = 1
    while cur != -1: path_idx.append(cur); cur = prev[cur]
    path_idx.reverse()
    return [nodes[i] for i in path_idx]
```

Computes the genuinely shortest path from source to destination that does not cut through the polygon, and returns it including both endpoints - so `path[1:-1]` are the via-points to insert.

The method is a **visibility graph**: build a graph whose nodes are the source, the destination and every polygon corner, join any two nodes whose connecting segment is not blocked, and run a shortest-path search over it. This is the classical construction for routing around polygonal obstacles, and it is **exact**, not a heuristic - the shortest obstacle-avoiding path always bends only at obstacle corners.

Dijkstra is implemented as a plain O(n squared) scan with no priority queue. With four marked corners the graph has six nodes, so the simple version is comfortably fast and avoids a dependency.

The `best[1] == math.inf` fallback returns the direct segment if no path was found at all, which can only happen for a degenerate polygon. The final loop walks the `prev` chain backwards from the destination and reverses it to recover the path.

Rewriting the route

_strip_safe_routes and _set_direct_route_only

```
def _strip_safe_routes(problem_text: str) -> str:
    return re.sub(r"[ \t]*\(safe-route\s+\S+\s+\S+\)\n?", "", problem_text)

def _set_direct_route_only(problem_text, source_name, destination_name) -> str:
    stripped = _strip_safe_routes(problem_text)
    addition = (f"\n        (safe-route {source_name} {destination_name})"
                f"\n        (safe-route {destination_name} {source_name})\n")
    matches = list(_CONNECTED_FACT.finditer(stripped))
    if not matches: return stripped + addition
    insert_at = matches[-1].end()
    return stripped[:insert_at] + addition + stripped[insert_at:]
```

The 'nothing is in the way' path. Every existing `safe-route` fact is deleted and exactly one pair - the direct leg, both directions - is added back after the last `connected` fact.

The docstring gives the reasoning, and it is the key design decision of the whole no-fly feature: ENHSP's default search is **satisficing**, not optimal. If both the direct leg and the template's detour were left legal, the planner would be free to return either, and the same two clicks could produce different routes on different runs. Removing the alternative outright - rather than merely making it more expensive - makes the outcome deterministic.

_set_detour_route(...)

```
def _set_detour_route(problem_text, source_name, destination_name,
                      new_source, new_destination, via_points) -> str:
    result = _strip_safe_routes(problem_text)
    if not via_points: return result
    names = [f"detour{i+1}" for i in range(len(via_points))]
    result = _add_locations(result, names)
    chain_names = [source_name] + names + [destination_name]
    chain_coords = [new_source] + via_points + [new_destination]
    facts = []
    for name, coord in zip(names, via_points):
        facts.append(f"\n        (= (latitude {name}) {coord.lat:.7f})")
        facts.append(f"\n        (= (longitude {name}) {coord.lon:.7f})")
    for a_name, a_pt, b_name, b_pt in zip(chain_names, chain_coords,
                                          chain_names[1:], chain_coords[1:]):
        distance_m = haversine_m(a_pt, b_pt)
        energy = distance_m * DEFAULT_ENERGY_RATE
        for x, y in ((a_name, b_name), (b_name, a_name)):
            facts.append(f"\n        (connected {x} {y})")
            facts.append(f"\n        (safe-route {x} {y})")
            facts.append(f"\n        (= (distance {x} {y}) {distance_m:.2f})")
            facts.append(f"\n        (= (energy-required {x} {y}) {energy:.2f})")
    return _add_init_facts(result, "".join(facts))
```

The 'something is in the way' path, and the counterpart to `_set_direct_route_only`. It makes the computed detour the **only** legal route.

Each via-point returned by `_shortest_detour` becomes a brand-new location object named `detour1`, `detour2`, and so on, with its own coordinate facts. The chain source-to-detour1-to-...-to-destination is then wired up with a full set of `connected`, `safe-route`, `distance` and `energy-required` facts in **both** directions.

Distances come from `haversine_m` on the real computed points, and energy from the module default rate of 0.0125 units per metre - these legs are freshly invented, so unlike the template's own edges they have no authored rate to preserve.

retarget_problem(...) - the main entry point [the most important function in the module]

```
def retarget_problem(problem_text, new_source, new_destination, *,
                     no_fly_zone=None, source_name="source",
                     destination_name="destination") -> str:
    locations = read_locations(problem_text)
    if source_name not in locations or destination_name not in locations:
        raise ValueError(...)

    transform = _similarity_transform(locations[source_name],
                                      locations[destination_name],
                                      new_source, new_destination)
    moved = {name: transform(point) for name, point in locations.items()}
    moved[source_name] = new_source              # pin the endpoints exactly
    moved[destination_name] = new_destination

    result = problem_text
    for name, point in moved.items():
        result = _set_latlon(result, name, point)

    for a, b in _read_edges(problem_text):
        if a not in moved or b not in moved: continue
        old_distance = _fact_value(problem_text, "distance", a, b)
        old_energy = _fact_value(problem_text, "energy-required", a, b)
        rate = (old_energy / old_distance
                if old_distance and old_energy is not None else DEFAULT_ENERGY_RATE)
        new_distance = haversine_m(moved[a], moved[b])
        if old_distance is not None:
            result = _set_fact_value(result, "distance", a, b, new_distance)
        if old_energy is not None:
            result = _set_fact_value(result, "energy-required", a, b, new_distance * rate)

    blocked = False; via_points = []
    if no_fly_zone and len(no_fly_zone) >= 3:
        seg_a = _to_local(new_source, new_source)            # (0, 0)
        seg_b = _to_local(new_destination, new_source)
        polygon_local = _convex_hull([_to_local(p, new_source) for p in no_fly_zone])
        endpoint_inside = (_point_in_polygon(seg_a, polygon_local)
                           or _point_in_polygon(seg_b, polygon_local))
        if len(polygon_local) >= 3 and not endpoint_inside:
            routing_polygon = _offset_polygon(polygon_local, NO_FLY_BUFFER_M)
            if (_point_in_polygon(seg_a, routing_polygon)
                    or _point_in_polygon(seg_b, routing_polygon)):
                routing_polygon = polygon_local          # buffer swallowed an endpoint
            blocked = _segment_crosses_polygon(seg_a, seg_b, routing_polygon)
            if blocked:
                detour_local = _shortest_detour(seg_a, seg_b, routing_polygon)
                via_points = [_from_local(p, new_source) for p in detour_local[1:-1]]

    if not blocked:
        result = _set_direct_route_only(result, source_name, destination_name)
    elif via_points:
        result = _set_detour_route(result, source_name, destination_name,
                                   new_source, new_destination, via_points)
    return result
```

Four phases, in order: move every location, re-cost every edge, decide whether the mission is blocked, and rewrite the legal routes accordingly.

Phase 1 - move. The transform is applied to every location, then the two endpoints are overwritten with the picked points directly. That pinning is not redundant: it avoids trusting a floating-point round trip to land exactly on the coordinates the operator clicked.

Phase 2 - re-cost. For each edge, the **original** energy-per-metre rate is recovered by dividing the authored energy by the authored distance, and the new energy is the new distance times that same rate. This is what preserves the authored asymmetry discussed in section 3 - the direct leg stays about twice as expensive per metre after the move. Edges with no authored cost fall back to `DEFAULT_ENERGY_RATE`.

Phase 3 - decide. Note what is **not** here: the template's own `(no-fly-zone waypoint1)` fact is ignored entirely. The docstring explains why at length - that waypoint is carried by the same proportional transform as everything else, so its distance from the new line scales with the length of the hop, making 'blocked' depend on how far apart

the clicks were rather than on any real obstruction. A restriction must be an explicit input.

Two guards sit inside phase 3. If either endpoint is **inside** the marked area, no detour is attempted at all - routing around a zone you are departing from or arriving in is meaningless. And if the 10 m buffer would swallow an endpoint the operator placed close to the zone, the code falls back to the true, unbuffered boundary, because a point inside the grown polygon could never be routed clear of it.

Phase 4 - rewrite. Exactly one of the two route writers runs, so precisely one route is ever legal. The remaining case - blocked, but no via-points came back, which only a degenerate polygon can produce - falls through both branches and leaves the template's own detour facts as the fallback.

Swarm expansion

_clone_named_lines and _clone_drone1_lines

```
def _clone_named_lines(text, span, token, extra_names) -> str:
    token_re = re.compile(rf"\b{re.escape(token)}\b")
    _open, close_pos = span
    clones = []
    for line in text[_open:close_pos].splitlines():
        if line.lstrip().startswith(";"): continue
        if token_re.search(line):
            clones.extend(token_re.sub(name, line) for name in extra_names)
    if not clones: return text
    head = text[:close_pos].rstrip()
    addition = "".join("\n" + line for line in clones)
    return f"{head}{addition}\n    {text[close_pos:]}"

def _clone_drone1_lines(text, span, extra_names) -> str:
    return _clone_named_lines(text, span, "drone1", extra_names)
```

The workhorse behind every swarm expansion in the codebase - Travell's, the V's and the grid's all route through it. It duplicates every **line** inside a span that mentions a token, once per new name, substituting the name in.

Working line-by-line is what makes the skipping of comment lines essential, and the docstring is explicit about the failure it prevents: an illustrative comment - an ASCII diagram, say - can split a parenthetical across two lines where only one mentions the token. Cloning that single line would inject an unmatched opening parenthesis and throw off every later `_find_matching_paren` call. Comments are not per-drone data anyway.

Line-orientation is also the exact weakness that caused the Grid-Formation bug documented in `PLANS_STATUS.txt`: a fact written across three lines had only its middle line cloned, producing a valueless fluent. The fix was to write that fact on one line, matching the convention every other per-drone fluent already used.

expand_point_to_point_drones(problem_text, count)

```
def expand_point_to_point_drones(problem_text: str, count: int) -> str:
    if count <= 1: return problem_text
    all_names = [f"drone{i}" for i in range(1, count + 1)]
    extra_names = all_names[1:]

    # 1. widen "drone1 - drone" -> "drone1 drone2 ... droneN - drone"
    obj_open, obj_close = _section_span(problem_text, "objects")
    widened = re.sub(r"\bdrone1\b(\s*-\s*drone\b)",
                     lambda m: " ".join(all_names) + m.group(1),
                     problem_text[obj_open:obj_close], count=1)
    if widened == problem_text[obj_open:obj_close]:
        raise ValueError("point-to-point template has no 'drone1 - drone' object to expand")
    problem_text = problem_text[:obj_open] + widened + problem_text[obj_close:]

    # 2. clone every (:init ...) fact that references drone1
    problem_text = _clone_drone1_lines(
        problem_text, _section_span(problem_text, "init"), extra_names)

    # 3. clone the goal's per-drone conjuncts, inside its (and ...)
    goal_open, goal_close = _section_span(problem_text, "goal")
    and_match = re.search(r"\(and\b", problem_text[goal_open:goal_close])
    if and_match:
        and_open = goal_open + and_match.start()
        span = (and_open, _find_matching_paren(problem_text, and_open))
    else:
        span = (goal_open, goal_close)
    problem_text = _clone_drone1_lines(problem_text, span, extra_names)
    return problem_text
```

Grows the single-drone template into an N-drone swarm, all identical and all flying the same planned route to the same destination.

Step 1 widens the object declaration in place, turning `drone1 - drone` into `drone1 drone2 ... droneN - drone`. The equality check afterwards is a real guard: if the substitution changed nothing, the template does not have the expected declaration, and failing loudly here is far better than producing a file that references undeclared objects.

Step 2 clones every initial-state line mentioning `drone1` - position, battery, health, GPS and comms flags - so each new drone starts identically.

Step 3 is the subtle one. The goal conjuncts must be cloned **inside** the goal's `(and ...)`, not merely inside `(:goal ...)`, or the result would be several sibling expressions where PDDL expects exactly one - a malformed goal. The code therefore locates the `(and` and uses its matching parenthesis as the insertion span, with a fallback for a goal that has no `and` at all.

The function is purely additive over the token `drone1`: the waypoint graph, routes and coordinates are untouched, which is why `_run` retargets **before** calling it.

4.3 engine/pddl_planner.py - running ENHSP and parsing the plan

The two exception types and PlanStep

```
ENHSP_ROOT = Path(__file__).resolve().parent.parent / "tools" / "enhsp"
DEFAULT_JAR = ENHSP_ROOT / "enhsp-dist" / "enhsp.jar"

class PlannerError(RuntimeError):
    """ENHSP could not be run at all (missing jar, missing Java, timeout)."""

class PlanNotFound(RuntimeError):
    """ENHSP ran, but the problem has no plan (infeasible route/battery)."""
    def __init__(self, message: str, stdout: str = ""):
        super().__init__(message)
        self.stdout = stdout

@dataclass(frozen=True)
class PlanStep:
    index: float
    action: str
    args: list[str]
    def __str__(self) -> str:
        return f"{self.index:g}: ({self.action} {' '.join(self.args)})"
```

The exception split is the module's most important design decision. `PlannerError` means **the tool did not run** - no jar, no Java, a timeout; something is wrong with the installation. `PlanNotFound` means **the tool ran and there is no plan** - the mission itself is infeasible. The GUI can therefore tell an operator 'this route is impossible' rather than 'planning failed'.

`PlanNotFound` carries ENHSP's raw `stdout`, which is what lets `_Worker.run` persist the planner's reasoning for a failed run.

`PlanStep` is one action: its time index, its name, and its arguments. It is frozen, so a parsed plan cannot be mutated downstream.

find_enhsp_jar() and _find_java()

```
def find_enhsp_jar() -> Path:
    override = os.environ.get("ENHSP_JAR")
    if override:
        path = Path(override)
        if not path.is_file():
            raise PlannerError(f"ENHSP_JAR is set but no jar exists at {path}")
        return path
    if not DEFAULT_JAR.is_file():
        raise PlannerError(f"ENHSP jar not found at {DEFAULT_JAR}. Build it with "
                           f"scripts/setup_enhsp.ps1, or set the ENHSP_JAR "
                           f"environment variable ...")
    return DEFAULT_JAR

def _find_java() -> str:
    import shutil
    on_path = shutil.which("java")
    if on_path: return on_path
    for base in (Path(os.environ.get("ProgramFiles", r"C:\Program Files")) / "Eclipse Adoptium",
                 Path(os.environ.get("ProgramFiles", r"C:\Program Files")) / "Java"):
        if not base.is_dir(): continue
        for candidate in sorted(base.glob("jdk-*"), reverse=True):
            exe = candidate / "bin" / "java.exe"
            if exe.is_file(): return str(exe)
    raise PlannerError("Could not find a `java` executable. Install a JDK (15+) ...")
```

Locating the two external dependencies. Both raise `PlannerError` with a message that says how to fix the problem, not merely that one occurred.

`_find_java` has a Windows-specific fallback worth understanding. Installing a JDK through `winget` updates the system PATH, but a process that was **already running** keeps the old environment - so `shutil.which("java")` can fail on a machine where Java is genuinely installed. The fallback looks in the standard Temurin and Java install directories and picks the highest-numbered JDK, sorting in reverse.

parse_plan(stdout)

```
_PLAN_LINE = re.compile(r"^\s*(?:[\d.]+\s*:\s*)?\(([^)]+)\)\s*$")

def parse_plan(stdout: str) -> list[PlanStep]:
    steps: list[PlanStep] = []
    for line in stdout.splitlines():
        match = _PLAN_LINE.match(line)
        if not match: continue
        index_text = line.split(":", 1)[0].strip()
        try:
            index = float(index_text)
        except ValueError:
            index = float(len(steps))
        body = match.group(1).split()
        if not body: continue
        steps.append(PlanStep(index=index, action=body[0], args=body[1:]))
    return steps
```

Extracts the ordered action list from ENHSP's console output, which interleaves the plan with grounding statistics, heuristic values and timings.

The whole design rests on one regex anchored with `^` and `$`: a plan line is a line that is **entirely** a parenthesised body, optionally prefixed by a numeric time index. Log lines such as `Plan-Length:9` or `h(I):25.0` cannot match that shape, so they are excluded by construction rather than by a blacklist. This makes the parser indifferent to which heuristic or search strategy ENHSP used and how verbose it was - which matters, because different plans in this project pass different flags.

The `try/except` around the index falls back to positional numbering if a line carries no parsable time, so a plan in an unindexed format still keeps its order.

run_enhsp(domain, problem, ...)

```
def run_enhsp(domain: Path, problem: Path, *, timeout_s: float = 30.0,
              extra_args: Optional[list[str]] = None) -> tuple[list[PlanStep], str]:
    jar = find_enhsp_jar()
    java = _find_java()
    args = [java, "-jar", str(jar), "-o", str(domain), "-f", str(problem)]
    if extra_args: args.extend(extra_args)
    try:
        result = subprocess.run(args, capture_output=True, text=True, timeout=timeout_s)
    except subprocess.TimeoutExpired as exc:
        raise PlannerError(f"ENHSP timed out after {timeout_s:.0f}s") from exc
    except OSError as exc:
        raise PlannerError(f"Could not launch ENHSP: {exc}") from exc
    stdout = result.stdout or ""
    steps = parse_plan(stdout)
    if not steps:
        detail = (result.stderr or stdout or "no output").strip()[-2000:]
        raise PlanNotFound(f"ENHSP found no plan for {problem.name}:\n{detail}", stdout=stdout)
    return steps, stdout
```

The subprocess call that does the actual planning. It builds the command line, runs Java to completion under a timeout, and parses the result.

Travell uses the defaults: a 30-second budget and no extra flags. The other plans override both - the Search plan passes a greedy best-first search with helpful actions, and Grid-Formation passes `-gro naive` to avoid a grounding blow-up. Making these parameters rather than constants is what lets one wrapper serve four quite different planning problems.

The empty-plan check is where the two exception types are assigned. Note the asymmetry: a timeout is a `PlannerError` because it says nothing about whether a plan exists, while producing no parsable steps is a `PlanNotFound`. The last 2000 characters of output are attached so the message carries ENHSP's own explanation.

extract_route(steps, drone)

```
def extract_route(steps: list[PlanStep], drone: str) -> list[str]:
    legs = [step.args[1:3]
            for step in sorted(steps, key=lambda s: s.index)
            if step.action == "travel" and step.args and step.args[0] == drone]
    if not legs: return []
    route = [legs[0][0]]
    route.extend(to for _from, to in legs)
    return route
```

Turns a plan back into a route. It keeps only the `travel` actions belonging to the named drone, sorted by time index, and takes each one's `?from`/`?to` pair.

The chaining is the neat part. Consecutive legs for one drone necessarily share a boundary - a drone that has travelled to `waypoint1` can only depart from `waypoint1` - so the route is simply the first leg's origin followed by every leg's destination. No deduplication or stitching is needed.

This is the function that makes `PLAN_DRONE_OBJECT = "drone1"` meaningful: in an 8-drone Travell run all eight drones have their own `travel` sequences in the plan, but since they are identical clones flying one shared route, reading `drone1`'s is enough.

The module's two other extractors, `extract_multi_route` and `extract_formation_corridor`, belong to the Search and formation plans respectively and are documented in those PDFs.

5. How the drones communicate

Communication appears at two quite different levels in this system, and it is worth being precise about which is which - they are not the same thing and they do not have the same topology.

5.1 In the PDDL model: a single link to the ground controller

Travell's communication model is the simplest of the four plans, because it has **no drone-to-drone communication at all**. Each drone carries exactly one communications fact:

The whole of Travell's communications vocabulary

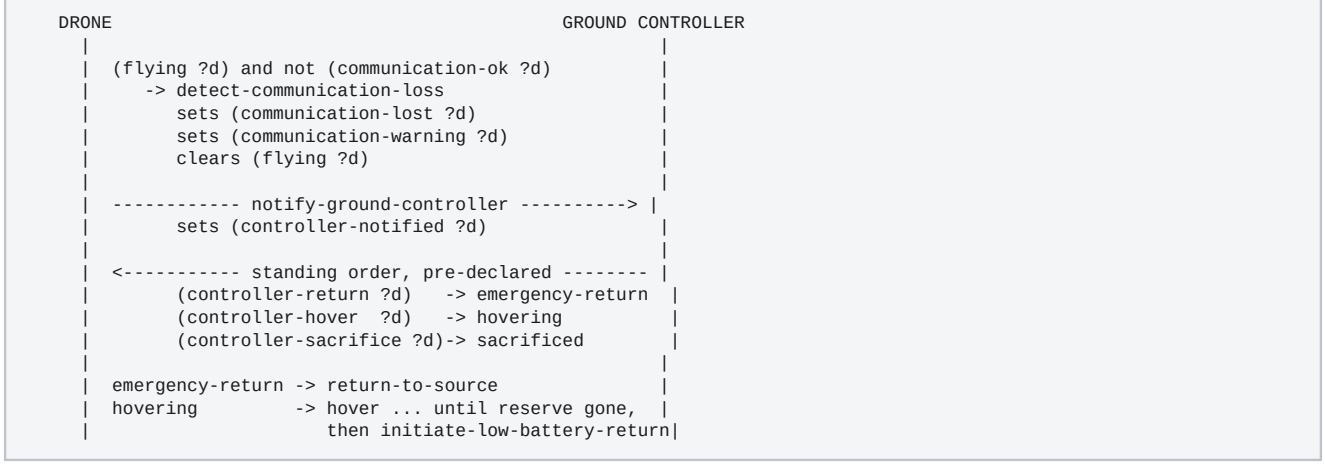
```
(communication-ok  ?d - drone)     ;; healthy radio link to the ground controller
(communication-lost ?d - drone)    ;; the link has been detected as down
(communication-warning ?d - drone) ;; telemetry flag raised alongside the loss
```

`communication-ok` is a link to the **ground controller**, not to another drone. It is checked as a precondition in exactly two places - `plan-safe-route` and `travel` - which together mean a drone that has lost its link may neither be given a new leg nor fly one. It cannot be repaired by any action; once gone it stays gone.

The two formation plans add a genuine neighbour model on top of this - `wing-neighbor`/`grid-neighbor` pairs and a per-drone `neighbor-comm-ok` flag. Travell has neither. Its drones are independent aircraft that happen to share a route; none of them needs to know that any of the others exist, and nothing in the domain couples one drone's state to another's.

5.2 The message loop, as the domain models it

The fault-report / order cycle in the Travell domain



Read upward first: all three detectors (`detect-gps-loss`, `detect-communication-loss`, `detect-health-failure`) funnel into the single `notify-ground-controller` action, which is the only upward message in the domain.

The downward direction is modelled differently, and this is a genuine modelling limitation worth stating in a write-up. The three orders are **not** actions the controller takes - they are facts that must already be true in the initial state. So the model expresses a **standing order** declared before launch ('if this drone gets in trouble, bring it home'), not a decision made in response to the report. In the shipped template none of the three is set, so all the emergency branches are reachable but dormant.

One consequence noted earlier bears repeating: `notify-ground-controller` does not require `communication-ok`, so a drone that has just lost its link can still report that fact. The model treats reporting as best-effort.

5.3 At runtime: a star topology through the GCS

When the planned route is actually flown, communication is handled by an entirely separate layer - `engine/protocol.py`, `engine/drone_link.py`, `engine/gcs.py` and `engine/fleet.py`. This layer knows nothing about PDDL.

The runtime transport

```
                    GROUND CONTROL STATION
                      (engine/gcs.py)
                       one UDP socket
                        port 14690
                            |
       +--------------------+--------------------+
       |                    |                    |
    udp:14701            udp:14702            udp:1470N
    Drone SYSID 1        Drone SYSID 2        Drone SYSID N
  (engine/drone_link.py)

  GCS  ------ COMMAND (JSON/UDP) ------>  drone
  GCS  <-- TELEMETRY / HEARTBEAT / ACK --  drone

  drone  <--------- X ---------->  drone      (no such link exists)
```

Every drone binds its own UDP port derived from its MAVLink SYSID - `drone_port(sysid) = 14700 + sysid` - so the GCS needs no demultiplexing on the way down. Every drone reports back to the single GCS socket, and every message carries `drone_id`, so one socket can address and sort a whole swarm. The wire format is one JSON datagram per message, chosen deliberately: it is the transport that later gets swapped for MAVLink, and an explicit schema makes that a translation exercise rather than a rewrite.

The important structural point, and the one to state plainly in any write-up: **this is a star, not a mesh**. There is no drone-to-drone socket anywhere in the codebase. Any coordination between drones - including the neighbour links the two formation plans model in PDDL - is mediated by the GCS, which holds the latest telemetry and heartbeat for every drone and fans commands back out. `DroneFleet.broadcast` and `GroundControlStation.broadcast` send the same command to many drones; they do not let drones talk to each other.

Link loss is therefore **observable** rather than assumed: the GCS tracks `last_heartbeat_at` per drone against a `HEARTBEAT_TIMEOUT_S` of 3 seconds, with heartbeats sent every second and telemetry every half second. That timeout is the runtime counterpart of the PDDL model's `communication-ok` flag going false.

6. A worked run

This is the tail of a real 8-drone Travell run from `plans/travell/runs/`, on a pick where nothing blocked the direct line.

ENHSP output - 8 drones, direct route

```
Problem Solved
Found Plan:
0.0: (check-distance drone6 source waypoint1)
1.0: (check-distance drone2 source waypoint1)
2.0: (check-distance drone7 source destination)
...
8.0: (check-battery drone5 source waypoint1)
...
16.0: (plan-safe-route drone2 source destination)
17.0: (plan-safe-route drone6 source destination)
...
24.0: (travel drone2 source destination)
25.0: (travel drone5 source destination)
26.0: (travel drone3 source destination)
27.0: (travel drone1 source destination)
...
```

Three things are worth reading out of this trace.

The three-phase rhythm is visible. All eight drones complete their distance checks, then their battery checks, then their route planning, then they travel. That layering is not scheduled by anyone - it falls out of the `distance-checked` to `battery-checked` to `route-planned` precondition chain in the domain.

`check-distance` mentions `waypoint1`, but `plan-safe-route` and `travel` do not. The waypoints still exist as objects and are still `connected`, so `check-distance` - which only requires `connected` - can be grounded against them and ENHSP happens to pick some of those groundings. But `_set_direct_route_only` stripped every `safe-route` except the direct pair, so the only leg that can actually be planned and flown is source-to-destination. This is exactly the `connected` versus `safe-route` distinction from section 2.3 doing its job.

The drone order is arbitrary. drone6 checks before drone2, drone2 travels before drone1. The drones are identical clones and the planner has no reason to prefer one ordering, so it returns whatever its search found first. Since `extract_route` reads only `drone1`'s legs and all eight routes are identical, the ordering is irrelevant to the result.

6.1 Record to date

Metric	Value
Runs logged	83
Solved	76
Unsolvable / no plan	7
Open code issues	None

The seven failures are all early testing runs from 2026-09-02, where a particular combination of picked route and marked restricted area genuinely left no safe path. Those are correct results, not defects: the planner reporting an infeasible click as infeasible.

7. Summary

- **The PDDL files are a template, never the flown mission.** Every run rewrites them onto the operator's picked points and writes the result into a timestamped `runs/` folder.
- **The domain name selects the behaviour.** `swarm-drone-mission` maps to `point_to_point` in `_DOMAIN_KIND`; the folder name is only a GUI label.

- *`connected` versus `safe-route` is the central trick.* Legs always exist; only some are legal, and the Python layer decides which by stripping and rewriting `safe-route` facts.
- **Exactly one route is ever legal**, because ENHSP's search is satisficing rather than optimal and leaving two options would make the outcome non-deterministic.
- **The detour is computed, not stored.** A visibility graph over the buffered polygon's corners plus Dijkstra gives the exact shortest path around a marked area.
- **Expansion is textual cloning** of every line mentioning `drone1`, in the objects block, the initial state, and inside the goal's `(and ...)`.
- **Communication in Travell is drone-to-ground only.** There is no peer link in the model, and no peer socket at runtime - the transport is a star through the GCS.